

Runway

A treasury pays people every second.

This decides who keeps getting paid when it runs short — and never holds the keys.

Finalist · 10 of 139 submissions

The problem

Superfluid announces nothing while the money drains.

A treasury pays salaries as streams. Money leaves the account every second — and there is **no event, no log to index, no threshold crossing** to watch.

- When the available balance reaches zero, **anyone may liquidate the sender** and take the locked buffer.
- **Every stream from that treasury collapses in the same block.** Not the smallest first, not the least important — all of them, at once.
- Payroll stops for everyone, with no warning, and the first person to notice is the one who was not paid.

The design decision everything rests on

The agent holds a mandate, not the keys.

Runway never holds the treasury's private key and **cannot be given it**. It operates through Superfluid's own access-control list — granted in one transaction, revoked in one transaction, enforced by the CFA contract rather than by Runway's code.

permissions = 6

Update and delete only — deliberately not 7. The contract reverts a **createFlow** from this operator. It can throttle or close a stream; it can never open one to an address of its own choosing.

A flow-rate allowance

Set to the sum of the committed rates. It can restore what was agreed and cannot exceed it.

Revocable without us

The treasury calls the same function with **permissions = 0**. Runway's cooperation is not required, and that is the point.

What one tick does

● Read

Balance, deposit, and every stream's live rate, off the chain. If any read fails, the tick fails — **it never decides on partial facts.**

● Decide

A pure function of those facts and a policy the treasury owns. Shed discretionary first, then standard, then critical — **each stopping at its floor.**

● Execute

One `superfluid/update-flow` per adjusted stream through KeeperHub, every parameter literal, each carrying an idempotency key derived from the work rather than from the attempt.

● Verify

Read every adjusted stream back. **A mined receipt proves a transaction landed; only a read proves the stream changed rate.**

● Escalate

On an incomplete read, a rejected mandate, an unknown write outcome, or a budget the floors cannot meet. **Silence is never the answer to a case the policy does not cover.**

It has actually run · Ethereum Sepolia, 8 September 2026

Runway fell from 214 hours to 23. Three streams, three transactions.

Stream	Rate before	Rate after	Outcome
Discretionary	57 860 765 330	11 572 153 066	read back at its floor
Standard	86 791 147 994	21 697 786 998	read back at its floor
Critical	144 651 913 324	86 791 147 994	read back at its floor

Wei per second, so the numbers are exact rather than rounded. All three landed and all three rates were read back off the chain. The floors still could not close 79 664 745 201 wei/sec, so **a floors-exceed-budget escalation was raised for the remainder** — the agent stopped at the floors instead of choosing who to cut.

Why you can believe those numbers

The evidence document is **parsed**, not written.

`pnpm test:evidence` reads every hash, link, block number and rate back out of the captured run record, and **fails if the document disagrees**.

- **The gate shipped red.** The document carried a sentinel no real value could equal, so no number could be written into it before a real run produced one.
- That is the difference between a figure in a README and a figure someone can check — and it is the only reason a reader should trust the slide before this one.

What it refuses to do

Guess, choose for you, or go quiet.

- **Reads fail closed.** A partial read raises an error listing every failure. A treasury keeper that guesses when the chain is unreachable is worse than one that stops.
- **It never cuts below a floor.** When the floors together still exceed the budget, it does not decide who matters least — it stops and hands the remainder to a human.
- **An escalation that was not delivered is recorded as not delivered.** Posted to the policy's webhook, retried with a widening delay under a hard timeout, and the delivery outcome written down either way.
- **It cannot pay anyone new.** Not as a policy — as a contract permission it was never granted.

Built on

Superfluid for the mandate. KeeperHub for the execution.

The run is recorded as JSON and rendered to a self-contained HTML report — no scripts, no remote resources, everything escaped. **The record is the product as much as the decision is.**

github.com/Yoh5/runway

the first transaction, on Sepolia